

Data Structures

TCS NQT - Core CS Subjects

TCS NQT Preparation Material
Generated on: April 05, 2026

Section 1: Arrays & Strings

Array operations, time complexities, string manipulation, and common interview problems.

Q1. [Easy]

What is the time complexity of accessing an element by index in an array?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Answer: A) $O(1)$

Explanation: Arrays provide $O(1)$ random access because elements are stored in contiguous memory and accessed via base address + offset.

Q2. [Easy]

What is the time complexity of inserting an element at the beginning of an array of size n ?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Answer: C) $O(n)$

Explanation: Inserting at the beginning requires shifting all n elements one position to the right, which is $O(n)$.

Q3. [Easy]

What is the worst-case time complexity of searching for an element in an unsorted array?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: C) $O(n)$

Explanation: In an unsorted array, you must check each element sequentially (linear search) = $O(n)$.

Q4. [Medium]

In a 2D array stored in row-major order, the address of element $A[i][j]$ is:

- A)** $\text{Base} + (i * \text{cols} + j) * \text{size}$
- B)** $\text{Base} + (j * \text{rows} + i) * \text{size}$
- C)** $\text{Base} + (i + j) * \text{size}$
- D)** $\text{Base} + (i * j) * \text{size}$

Answer: A) $\text{Base} + (i * \text{cols} + j) * \text{size}$

Explanation: Row-major: elements of a row are contiguous. $\text{Address} = \text{Base} + (i * \text{number_of_columns} + j) * \text{element_size}$.

Q5. [Easy]

What is the space complexity of a string of length n ?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: C) $O(n)$

Explanation: A string of length n requires $O(n)$ space to store n characters.

Q6. [Medium]

What is the time complexity of concatenating two strings of lengths m and n ?

- A)** $O(1)$
- B)** $O(m)$
- C)** $O(n)$
- D)** $O(m + n)$

Answer: D) $O(m + n)$

Explanation: Concatenation copies all characters from both strings into a new string, taking $O(m + n)$ time.

Q7. [Easy]

Which data structure is best for implementing a dynamic array?

- A)** Linked list
- B)** ArrayList / Vector
- C)** Stack
- D)** Queue

Answer: B) ArrayList / Vector

Explanation: ArrayList/Vector provides dynamic resizing while maintaining $O(1)$ amortized insertion at the end.

Q8. [Medium]

When a dynamic array doubles its capacity upon being full, the amortized time complexity of insertion is:

- A)** $O(n)$
- B)** $O(\log n)$
- C)** $O(1)$ amortized
- D)** $O(n^2)$

Answer: C) $O(1)$ amortized

Explanation: Though individual resize operations are $O(n)$, using amortized analysis the average cost per insertion is $O(1)$.

Q9. [Medium]

What is a sparse array?

- A)** An array with no elements
- B)** An array where most elements are zero or default
- C)** A sorted array
- D)** A multidimensional array

Answer: B) An array where most elements are zero or default

Explanation: A sparse array has most elements as default/zero. It can be stored efficiently using specialized representations like triplet form.

Q10. [Hard]

The KMP string matching algorithm has time complexity of:

- A)** $O(m * n)$
- B)** $O(m + n)$
- C)** $O(n)$
- D)** $O(n \log n)$

Answer: B) $O(m + n)$

Explanation: KMP preprocesses the pattern in $O(m)$ and searches in $O(n)$, giving $O(m + n)$ total where m = pattern length, n = text length.

Section 2: Linked Lists

Singly linked, doubly linked, circular linked lists, and their operations.

Q11. [Easy]

What is the time complexity of inserting a node at the beginning of a singly linked list?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: A) $O(1)$

Explanation: Inserting at the head requires creating a new node and updating the head pointer - constant time.

Q12. [Easy]

What is the time complexity of deleting the last node in a singly linked list?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: C) $O(n)$

Explanation: You must traverse the entire list to find the second-to-last node since there is no backward pointer.

Q13. [Easy]

In a doubly linked list, each node contains:

- A)** Data and one pointer
- B)** Data and two pointers (prev and next)
- C)** Only data
- D)** Three pointers

Answer: B) Data and two pointers (prev and next)

Explanation: A doubly linked list node has data, a pointer to the next node, and a pointer to the previous node.

Q14. [Easy]

What is the advantage of a doubly linked list over a singly linked list?

- A)** Uses less memory
- B)** Faster insertion at head
- C)** Can traverse in both directions
- D)** Simpler implementation

Answer: C) Can traverse in both directions

Explanation: The prev pointer allows traversal in both directions, making deletion of a given node $O(1)$ if you have a pointer to it.

Q15. [Easy]

In a circular linked list, the last node points to:

- A)** NULL
- B)** The first node
- C)** The middle node
- D)** Itself

Answer: B) The first node

Explanation: In a circular linked list, the last node's next pointer points back to the head, forming a loop.

Q16. [Medium]

How can you detect a cycle in a linked list?

- A)** Using two pointers (slow and fast) - Floyd cycle detection
- B)** Using recursion
- C)** Sorting the list
- D)** Reversing the list

Answer: A) Using two pointers (slow and fast) - Floyd cycle detection

Explanation: Floyd's algorithm uses a slow pointer (1 step) and fast pointer (2 steps). If they meet, a cycle exists. $O(n)$ time, $O(1)$ space.

Q17. [Easy]

What is the time complexity of searching for an element in a singly linked list?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Answer: C) $O(n)$

Explanation: Linked lists do not support random access, so searching requires traversing from head - $O(n)$ worst case.

Q18. [Medium]

Reversing a singly linked list in-place has time and space complexity of:

- A) $O(n)$ time, $O(n)$ space
- B) $O(n)$ time, $O(1)$ space
- C) $O(n^2)$ time, $O(1)$ space
- D) $O(\log n)$ time, $O(1)$ space

Answer: B) $O(n)$ time, $O(1)$ space

Explanation: Iterative reversal traverses the list once ($O(n)$) using three pointers (prev, current, next) - $O(1)$ extra space.

Q19. [Medium]

Which operation is MORE efficient in a linked list compared to an array?

- A) Random access
- B) Insertion/deletion at a known position
- C) Binary search
- D) Sorting

Answer: B) Insertion/deletion at a known position

Explanation: If you have a pointer to the position, insertion/deletion in a linked list is $O(1)$ vs $O(n)$ for arrays (due to shifting).

Q20. [Medium]

What is a sentinel node (dummy head) in a linked list?

- A)** The last node
- B)** A dummy node at the beginning that simplifies edge cases
- C)** A node that stores the length
- D)** A node with maximum value

Answer: B) A dummy node at the beginning that simplifies edge cases

Explanation: A sentinel node is a dummy node placed before the first element, eliminating the need for special-case handling of head.

Section 3: Stacks & Queues

Stack and queue implementations, applications, infix/postfix conversion, and related problems.

Q21. [Easy]

A stack follows which principle?

- A) FIFO
- B) LIFO
- C) FILO
- D) Both B and C

Answer: D) Both B and C

Explanation: A stack is Last In First Out (LIFO) which is the same as First In Last Out (FILO).

Q22. [Medium]

What is the postfix expression for the infix expression: $A + B * C$?

- A) ABC^{*+}
- B) $AB+C^{*}$
- C) $+A^{*}BC$
- D) $A+BC^{*}$

Answer: A) ABC^{*+}

Explanation: Following operator precedence: $B * C$ first, then $A + \text{result}$. Postfix: $A B C * +$.

Q23. [Easy]

Which data structure is used for function call management in programming?

- A) Queue
- B) Stack
- C) Array
- D) Tree

Answer: B) Stack

Explanation: The call stack stores function return addresses, local variables, and parameters in LIFO order.

Q24. [Easy]

A queue follows which principle?

- A) LIFO**
- B) FIFO**
- C) Random**
- D) Priority**

Answer: B) FIFO

Explanation: A queue is First In First Out (FIFO) - elements are added at the rear and removed from the front.

Q25. [Easy]

What is the time complexity of push and pop operations on a stack (array implementation)?

- A) $O(1)$, $O(1)$**
- B) $O(n)$, $O(n)$**
- C) $O(1)$, $O(n)$**
- D) $O(n)$, $O(1)$**

Answer: A) $O(1)$, $O(1)$

Explanation: Both push and pop operate on the top of the stack, requiring constant time.

Q26. [Medium]

A circular queue solves which problem of a linear queue?

- A) Overflow**
- B) Underflow**
- C) Wasted space due to front moving forward**
- D) Slow insertion**

Answer: C) Wasted space due to front moving forward

Explanation: In a linear queue, dequeued positions cannot be reused. A circular queue wraps around to reuse freed positions.

Q27. [Medium]

How can you implement a queue using two stacks?

- A) Not possible
- B) Push to stack1; for dequeue, move all from stack1 to stack2 and pop
- C) Use both stacks alternately
- D) Merge stacks

Answer: B) Push to stack1; for dequeue, move all from stack1 to stack2 and pop

Explanation: Enqueue: push to stack1. Dequeue: if stack2 is empty, transfer all from stack1 to stack2, then pop from stack2.

Q28. [Easy]

A priority queue dequeues elements based on:

- A) Insertion order
- B) Priority value
- C) Random order
- D) Size

Answer: B) Priority value

Explanation: A priority queue serves the highest (or lowest) priority element first, regardless of insertion order.

Q29. [Easy]

Which application uses a stack?

- A) BFS traversal
- B) Expression evaluation and parenthesis matching
- C) CPU scheduling
- D) Printer spooling

Answer: B) Expression evaluation and parenthesis matching

Explanation: Stacks are used for expression evaluation, parenthesis matching, undo operations, backtracking, and DFS.

Q30. [Medium]

What is a deque (double-ended queue)?

- A)** A queue with two fronts
- B)** A structure allowing insertion and deletion at both ends
- C)** Two queues combined
- D)** A priority queue

Answer: B) A structure allowing insertion and deletion at both ends

Explanation: A deque allows $O(1)$ insertion and deletion at both front and rear ends.

Section 4: Trees

Binary trees, BST, AVL trees, heaps, traversals, and tree properties.

Q31. [Medium]

The maximum number of nodes in a binary tree of height h is:

- A) 2^h
- B) $2^{(h+1)} - 1$
- C) $2h + 1$
- D) h^2

Answer: B) $2^{(h+1)} - 1$

Explanation: A complete binary tree of height h (root at $h=0$) has at most $2^{(h+1)} - 1$ nodes.

Q32. [Easy]

In-order traversal of a Binary Search Tree gives:

- A) Random order
- B) Ascending sorted order
- C) Descending sorted order
- D) Level order

Answer: B) Ascending sorted order

Explanation: In-order traversal visits Left, Root, Right. For a BST, this produces elements in ascending order.

Q33. [Easy]

What is the time complexity of search in a balanced BST?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: B) $O(\log n)$

Explanation: A balanced BST has height $O(\log n)$, and search follows a path from root to leaf.

Q34. [Easy]

What is the worst-case time complexity of search in a skewed BST?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: C) $O(n)$

Explanation: A skewed BST degenerates into a linked list with height n , making search $O(n)$.

Q35. [Easy]

An AVL tree maintains balance by ensuring the height difference between left and right subtrees is at most:

- A)** 0
- B)** 1
- C)** 2
- D)** 3

Answer: B) 1

Explanation: AVL trees maintain a balance factor ($|\text{height}(\text{left}) - \text{height}(\text{right})|$) of at most 1 for every node.

Q36. [Medium]

Which rotation is used when a node is inserted into the right subtree of the right child (AVL)?

- A)** Left rotation
- B)** Right rotation
- C)** Left-Right rotation
- D)** Right-Left rotation

Answer: A) Left rotation

Explanation: RR imbalance is fixed by a single left rotation. LL by right rotation. LR and RL need double rotations.

Q37. [Easy]

A max-heap is a complete binary tree where:

- A)** Parent is smaller than children
- B)** Parent is greater than or equal to children
- C)** Left child is greater than right
- D)** All leaves are at the same level

Answer: B) Parent is greater than or equal to children

Explanation: In a max-heap, each parent node is greater than or equal to its children. The maximum element is at the root.

Q38. [Medium]

What is the time complexity of extracting the maximum element from a max-heap?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n \log n)$

Answer: B) $O(\log n)$

Explanation: Finding max is $O(1)$ (root), but after removing it, heapify-down takes $O(\log n)$ to restore the heap property.

Q39. [Easy]

Pre-order traversal visits nodes in which order?

- A)** Left, Root, Right
- B)** Root, Left, Right
- C)** Left, Right, Root
- D)** Root, Right, Left

Answer: B) Root, Left, Right

Explanation: Pre-order: Root, Left, Right. In-order: Left, Root, Right. Post-order: Left, Right, Root.

Q40. [Medium]

The number of leaf nodes in a full binary tree with n internal nodes is:

- A)** n
- B)** $n + 1$
- C)** $n - 1$
- D)** $2n$

Answer: B) $n + 1$

Explanation: In a full binary tree (every node has 0 or 2 children), the number of leaves = internal nodes + 1.

Q41. [Easy]

Level-order traversal of a tree uses which data structure?

- A)** Stack
- B)** Queue
- C)** Priority Queue
- D)** Array

Answer: B) Queue

Explanation: Level-order (BFS) traversal processes nodes level by level using a queue.

Q42. [Medium]

What is a complete binary tree?

- A)** Every node has 2 children
- B)** All levels are fully filled except possibly the last, which is filled from left
- C)** All leaves at same level
- D)** A binary tree with n nodes

Answer: B) All levels are fully filled except possibly the last, which is filled from left

Explanation: A complete binary tree fills levels left to right. This property allows efficient array representation (used in heaps).

Q43. [Hard]

The time complexity of building a heap from an unsorted array is:

- A)** $O(n)$
- B)** $O(n \log n)$
- C)** $O(n^2)$
- D)** $O(\log n)$

Answer: A) $O(n)$

Explanation: Bottom-up heap construction (heapify) runs in $O(n)$ time, which is more efficient than inserting elements one by one $O(n \log n)$.

Q44. [Medium]

In a BST, the successor of a node (in in-order traversal) is:

- A)** The parent node
- B)** The leftmost node in the right subtree
- C)** The rightmost node in the left subtree
- D)** The root node

Answer: B) The leftmost node in the right subtree

Explanation: The in-order successor is the smallest node greater than the current node, found as the leftmost node of the right subtree.

Q45. [Hard]

A B-tree of order m has the property that each non-root internal node has at least:

- A)** m children
- B)** $m/2$ children (ceiling)
- C)** m-1 children
- D)** 2 children

Answer: B) $m/2$ children (ceiling)

Explanation: In a B-tree of order m, non-root internal nodes have between $\text{ceil}(m/2)$ and m children.

Section 5: Graphs

BFS, DFS, shortest path algorithms, minimum spanning trees, and topological sorting.

Q46. [Easy]

BFS uses which data structure?

- A) Stack
- B) Queue
- C) Priority Queue
- D) Hash Table

Answer: B) Queue

Explanation: BFS (Breadth-First Search) explores level by level using a queue.

Q47. [Easy]

DFS uses which data structure?

- A) Queue
- B) Stack (or recursion)
- C) Priority Queue
- D) Linked List

Answer: B) Stack (or recursion)

Explanation: DFS (Depth-First Search) uses a stack (explicit or implicit via recursion) to explore as deep as possible first.

Q48. [Medium]

The time complexity of BFS and DFS on a graph with V vertices and E edges is:

- A) $O(V)$
- B) $O(E)$
- C) $O(V + E)$
- D) $O(V * E)$

Answer: C) $O(V + E)$

Explanation: Both BFS and DFS visit all vertices and edges once (using adjacency list), giving $O(V + E)$.

Q49. [Easy]

Dijkstra's algorithm finds:

- A) Minimum spanning tree
- B) Shortest path from a single source (non-negative weights)
- C) All pairs shortest path
- D) Topological order

Answer: B) Shortest path from a single source (non-negative weights)

Explanation: Dijkstra's is a greedy algorithm for single-source shortest paths. It fails with negative edge weights.

Q50. [Medium]

Which algorithm handles negative edge weights for shortest paths?

- A) Dijkstra
- B) Bellman-Ford
- C) Prim
- D) Kruskal

Answer: B) Bellman-Ford

Explanation: Bellman-Ford handles negative weights and can detect negative cycles. Time complexity: $O(V * E)$.

Q51. [Medium]

Kruskal's algorithm finds the MST by:

- A) Adding the nearest vertex
- B) Sorting edges by weight and adding them if no cycle is formed
- C) Using BFS
- D) Using DFS

Answer: B) Sorting edges by weight and adding them if no cycle is formed

Explanation: Kruskal's sorts all edges and greedily adds the smallest edge that does not form a cycle (using Union-Find).

Q52. [Hard]

Prim's algorithm for MST has time complexity with a binary heap of:

- A)** $O(V^2)$
- B)** $O(E \log V)$
- C)** $O(V * E)$
- D)** $O(V + E)$

Answer: B) $O(E \log V)$

Explanation: With a binary heap/priority queue, Prim's runs in $O(E \log V)$. With adjacency matrix (no heap): $O(V^2)$.

Q53. [Easy]

Topological sorting is possible only for:

- A)** Undirected graphs
- B)** Directed Acyclic Graphs (DAGs)
- C)** Weighted graphs
- D)** Complete graphs

Answer: B) Directed Acyclic Graphs (DAGs)

Explanation: Topological sort requires a DAG. A cycle makes it impossible since there is no valid linear ordering.

Q54. [Medium]

Floyd-Warshall algorithm finds:

- A)** Single source shortest path
- B)** All pairs shortest paths
- C)** Minimum spanning tree
- D)** Maximum flow

Answer: B) All pairs shortest paths

Explanation: Floyd-Warshall computes shortest paths between all pairs of vertices in $O(V^3)$ time using dynamic programming.

Q55. [Easy]

A graph with V vertices can have at most how many edges (undirected, no self-loops)?

- A)** V
- B)** $V - 1$
- C)** $V(V-1)/2$
- D)** V^2

Answer: C) $V(V-1)/2$

Explanation: An undirected graph without self-loops can have at most $C(V,2) = V(V-1)/2$ edges.

Q56. [Medium]

Which algorithm detects a cycle in an undirected graph?

- A)** Dijkstra
- B)** DFS or Union-Find
- C)** BFS only
- D)** Topological Sort

Answer: B) DFS or Union-Find

Explanation: DFS detects a cycle if a visited node (not the parent) is encountered. Union-Find detects a cycle if two vertices of a new edge are already in the same set.

Q57. [Easy]

A minimum spanning tree of a graph with V vertices has how many edges?

- A)** V
- B)** $V - 1$
- C)** $V + 1$
- D)** $E - 1$

Answer: B) $V - 1$

Explanation: A spanning tree of V vertices always has exactly $V - 1$ edges.

Section 6: Hashing

Hash functions, collision resolution techniques, and load factor analysis.

Q58. [Easy]

The average time complexity of search in a hash table is:

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: A) $O(1)$

Explanation: With a good hash function and low load factor, hash table operations are $O(1)$ on average.

Q59. [Easy]

Which collision resolution technique uses linked lists at each bucket?

- A)** Linear probing
- B)** Quadratic probing
- C)** Separate chaining
- D)** Double hashing

Answer: C) Separate chaining

Explanation: Separate chaining stores colliding elements in a linked list at the hash table index.

Q60. [Easy]

In linear probing, if a collision occurs at index h , the next index checked is:

- A)** $h + 1$
- B)** h^2
- C)** $h * 2$
- D)** Random

Answer: A) $h + 1$

Explanation: Linear probing checks $h+1$, $h+2$, $h+3...$ (mod table size) sequentially until an empty slot is found.

Q61. [Medium]

What is the primary clustering problem associated with?

- A) Separate chaining
- B) Linear probing
- C) Double hashing
- D) Quadratic probing

Answer: B) Linear probing

Explanation: Primary clustering in linear probing occurs when consecutive filled slots form clusters, increasing probe lengths.

Q62. [Easy]

The load factor of a hash table is:

- A) Number of elements / Table size
- B) Table size / Number of elements
- C) Number of collisions
- D) Hash function complexity

Answer: A) Number of elements / Table size

Explanation: Load factor $\alpha = n/m$ where n = number of elements, m = table size. Higher load factor means more collisions.

Q63. [Medium]

Double hashing uses:

- A) One hash function
- B) Two hash functions to determine probe sequence
- C) Three hash functions
- D) No hash function

Answer: B) Two hash functions to determine probe sequence

Explanation: Double hashing: $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$. The second hash function determines the step size.

Q64. [Medium]

When should a hash table be resized?

- A) When it is empty
- B) When load factor exceeds a threshold (typically 0.7)
- C) After every insertion
- D) Never

Answer: B) When load factor exceeds a threshold (typically 0.7)

Explanation: Resizing (rehashing) when load factor is high maintains $O(1)$ average performance. Common threshold is 0.7-0.75.

Q65. [Medium]

What is the worst-case time complexity of hash table operations?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Answer: C) $O(n)$

Explanation: Worst case occurs when all elements hash to the same bucket (all collisions), degenerating to a linked list: $O(n)$.

Section 7: Sorting & Searching

Comparison of sorting algorithms, binary search, and their time/space complexities.

Q66. [Easy]

What is the time complexity of binary search?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: B) $O(\log n)$

Explanation: Binary search halves the search space each step, requiring at most $\log_2(n)$ comparisons on a sorted array.

Q67. [Easy]

Which sorting algorithm has the best average-case time complexity?

- A) Bubble Sort - $O(n^2)$
- B) Selection Sort - $O(n^2)$
- C) Merge Sort - $O(n \log n)$
- D) Insertion Sort - $O(n^2)$

Answer: C) Merge Sort - $O(n \log n)$

Explanation: Merge Sort, Quick Sort, and Heap Sort all have $O(n \log n)$ average case. Among the options, Merge Sort is the best.

Q68. [Medium]

Which sorting algorithm is NOT stable?

- A) Merge Sort
- B) Insertion Sort
- C) Quick Sort
- D) Bubble Sort

Answer: C) Quick Sort

Explanation: Quick Sort is not stable (equal elements may be reordered). Merge Sort, Insertion Sort, and Bubble Sort are stable.

Q69. [Medium]

The worst-case time complexity of Quick Sort is:

- A) $O(n \log n)$
- B) $O(n^2)$
- C) $O(n)$
- D) $O(\log n)$

Answer: B) $O(n^2)$

Explanation: Quick Sort's worst case is $O(n^2)$ when the pivot is always the smallest or largest element (e.g., sorted input with first element as pivot).

Q70. [Medium]

Merge Sort has space complexity of:

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Answer: C) $O(n)$

Explanation: Merge Sort requires $O(n)$ extra space for the temporary arrays used during merging.

Q71. [Medium]

Which sorting algorithm is best for nearly sorted data?

- A) Quick Sort
- B) Merge Sort
- C) Insertion Sort
- D) Selection Sort

Answer: C) Insertion Sort

Explanation: Insertion Sort runs in $O(n)$ on nearly sorted data since very few shifts are needed.

Q72. [Medium]

Heap Sort has time complexity of:

- A)** $O(n)$ in all cases
- B)** $O(n \log n)$ in all cases
- C)** $O(n^2)$ worst case
- D)** $O(n \log n)$ average, $O(n^2)$ worst

Answer: B) $O(n \log n)$ in all cases

Explanation: Heap Sort guarantees $O(n \log n)$ time for best, average, and worst cases, with $O(1)$ extra space.

Q73. [Hard]

Counting Sort has time complexity of:

- A)** $O(n \log n)$
- B)** $O(n + k)$ where k is the range
- C)** $O(n^2)$
- D)** $O(n)$

Answer: B) $O(n + k)$ where k is the range

Explanation: Counting Sort is a non-comparison sort with $O(n + k)$ time. It works well when k (range of values) is small relative to n .

Section 8: Complexity Analysis

Big-O notation, recurrence relations, Master theorem, and asymptotic analysis.

Q74. [Easy]

What does $O(1)$ mean?

- A) Linear time
- B) Constant time
- C) Logarithmic time
- D) Quadratic time

Answer: B) Constant time

Explanation: $O(1)$ means the operation takes constant time regardless of input size.

Q75. [Easy]

Which of the following growth rates is the fastest?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(2^n)$
- D) $O(n^2)$

Answer: C) $O(2^n)$

Explanation: Order: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$.

Q76. [Medium]

The recurrence $T(n) = 2T(n/2) + n$ has the solution:

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$

Answer: B) $O(n \log n)$

Explanation: By Master theorem: $a=2$, $b=2$, $f(n)=n$. $n^{\log_b(a)} = n^1 = n$. Since $f(n) = \Theta(n)$, $T(n) = O(n \log n)$. This is Merge Sort.

Q77. [Medium]

The recurrence $T(n) = T(n/2) + 1$ solves to:

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: B) $O(\log n)$

Explanation: $a=1$, $b=2$, $f(n)=1$. $n^{(\log_2(1))} = n^0 = 1$. $f(n) = \Theta(1)$. $T(n) = O(\log n)$. This is binary search.

Q78. [Medium]

The Master Theorem applies to recurrences of the form:

- A) $T(n) = aT(n/b) + f(n)$
- B) $T(n) = T(n-1) + n$
- C) $T(n) = T(n-1) + T(n-2)$
- D) Any recurrence

Answer: A) $T(n) = aT(n/b) + f(n)$

Explanation: The Master Theorem solves divide-and-conquer recurrences where the problem is divided into a subproblems of size n/b .

Q79. [Medium]

What is the time complexity of $T(n) = T(n-1) + n$?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(2^n)$

Answer: C) $O(n^2)$

Explanation: $T(n) = n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 = O(n^2)$. This is like selection sort.

Q80. [Hard]

What is the time complexity of $T(n) = 2T(n-1) + 1$?

- A)** $O(n)$
- B)** $O(n^2)$
- C)** $O(2^n)$
- D)** $O(n \log n)$

Answer: C) $O(2^n)$

Explanation: This recurrence represents exponential growth. $T(n) = 2^n - 1 = O(2^n)$. Example: Tower of Hanoi.

Q81. [Hard]

Amortized analysis considers:

- A)** Worst case of a single operation
- B)** Average cost per operation over a sequence of operations
- C)** Best case only
- D)** Space complexity only

Answer: B) Average cost per operation over a sequence of operations

Explanation: Amortized analysis averages the cost over a sequence. Even if some operations are expensive, the average cost may be low (e.g., dynamic array resizing).